Assignment 5

Q1

Q1.java

Run ▶

```java
// Q1
import java.util.*;
public class Q1 {
    static <T extends Number> double sum(T[] arr) {
        double s = 0;
        for (T t : arr) s += t.doubleValue();
        return s;
    }
    public static void main(String[] args) {
        Integer[] i = {1, 2, 3, 4};
        Float[] f = {1.5f, 2.5f, 3.5f};
        Double[] d = {1.1, 2.2, 3.3};
        System.out.println(sum(i));
        System.out.println(sum(f));
        System.out.println(sum(d));
    }
}
```

Output

```
10.0
7.5
6.6
```

Q2

Q2.java

Run ▶

```java
// Q2
import java.util.*;

public class Q2 {
    public static void main(String[] args) {
        Optional<String> email = Optional.of("student@tiet.edu");
        Optional<String> empty = Optional.empty();

        email.ifPresent(e -> System.out.println("Email: " + e));
        System.out.println(empty.orElse("default@tiet.edu"));

        try {
            String e = empty.orElseThrow(() -> new
IllegalStateException("Email missing"));
            System.out.println(e);
        } catch (IllegalStateException ex) {
            System.out.println("Exception: " + ex.getMessage());
        }
    }
}
```

Output

```
Email: student@tiet.edu
default@tiet.edu
Exception: Email missing
```

Q3

Q3.java

Run

```java
// Q3
import java.util.*;
public class Q3 {
    public static void main(String[] args) {
        Result<Integer> r = new Result<>(Objects.requireNonNull(78), 100);
        r.getMarks().ifPresent(m -> System.out.println("Marks: " + m));
        System.out.println("Percentage: " + r.percentage());
        System.out.println("Pass: " + r.isPass(40));

        Result<Double> r2 = new Result<>(null, 100);
        System.out.println(r2.getMarks().isPresent() ? "Marks available" :
"Marks not available");
    }
}
class Result<T extends Number> {
    private T marks;
    private double total;
    Result(T marks, double total) {
        this.marks = marks;
        this.total = Objects.requireNonNull(total);
    }
    Optional<T> getMarks() {
        return Optional.ofNullable(marks);
    }
    double percentage() {
        return (marks.doubleValue() / total) * 100;
    }
    boolean isPass(double threshold) {
        return percentage() >= threshold;
    }
}
```

Output

```
Marks: 78
Percentage: 78.0
Pass: true
Marks not available
```

Q4

```java
// Q4
import java.time.*;
import java.time.temporal.ChronoUnit;
import java.util.*;
public class Q4 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Event name: ");
        String name = sc.nextLine();
        System.out.print("Event date (yyyy-mm-dd): ");
        LocalDate date = LocalDate.parse(sc.nextLine());
        System.out.print("Start time (HH:mm): ");
        LocalTime start = LocalTime.parse(sc.nextLine());
        System.out.print("Venue (press enter to skip): ");
        String v = sc.nextLine();
        Optional<String> venue = v.isEmpty() ? Optional.empty() : Optional.of(v);

        System.out.println("Event: " + name);
        System.out.println("Date: " + date);
        System.out.println("Start: " + start);
        System.out.println("Venue: " + venue.orElse("Venue not specified"));

        LocalDate today = LocalDate.now();
        if (date.isAfter(today)) System.out.println("Event is in the future");
        else if (date.isEqual(today)) System.out.println("Event is today");
        else System.out.println("Event is in the past");

        System.out.println("Days remaining: " + ChronoUnit.DAYS.between(today, date));
        System.out.println("End time: " + start.plusHours(2));
    }
}
```

Output

```
Event name: Tech fest
Event date (yyyy-mm-dd): 2026-11-12
Start time (HH:mm): 15:30
Venue (press enter to skip): Auditorium
Event: Tech fest
Date: 2026-11-12
Start: 15:30
Venue: Auditorium
Event is in the future
Days remaining: 19
End time: 17:30
```

Q5

```java
// Q5
import java.util.*;
public class Q5 {
    public static void main(String[] args) {
        System.out.println(Utility.isEqual("abc", "abc"));
        System.out.println(Utility.isEqual(10, 20));
        System.out.println(Utility.isEqual(2.5, 2.5));

        System.out.println(Utility.getDefault(Optional.of("Hello"), "Default"));
        System.out.println(Utility.getDefault(Optional.empty(), "Default"));
        System.out.println(Utility.getDefault(Optional.<Integer>empty(), 99));

        System.out.println(Utility.max("apple", "banana"));
        System.out.println(Utility.max(15, 9));
        System.out.println(Utility.max(3.5, 7.2));

        System.out.println(Utility.isNull(null));
        System.out.println(Utility.isNull("text"));
    }
}
class Utility {
    static <T> boolean isEqual(T a, T b) {
        return Objects.equals(a, b);
    }
    static <T> T getDefault(Optional<T> value, T defaultValue) {
        return value.orElse(defaultValue);
    }
    static <T extends Comparable<T>> T max(T a, T b) {
        return a.compareTo(b) >= 0 ? a : b;
    }
    static boolean isNull(Object obj) {
        return Objects.isNull(obj);
    }
}
```

Output

```
true
false
true
Hello
Default
99
banana
15
7.2
true
false
```